Rare Book Data Analysis

Richard Freedman

2025-11-15

Concert Program Pattern Analysis

Explores patterns across the structured concert-program data extracted in `RareBooks_DataExtraction.ipynb` and `code/Langchain_for_Rare_Books.ipynb`: conductors, composers, and performers across ensembles, and how they change over time.

- **Primary source:** `Structured Data/concert_programs_events.json` — one row per concert, with a free-form `credits` dict (role → person) and a `Composer (<work title>)` → composer entry for each piece performed.
- One question (violinists over time for the Melbourne Liedertafel) needs performer-level rosters that this file doesn't capture for that ensemble, so that section instead reads `Structured Data/concert_program_items.json`, the flat item export from `RareBooks_DataExtraction.ipynb`. Noted again at that section.

All charts use Plotly Express.

0. Setup

```
from pathlib import Path
import json
import re

import pandas as pd
import plotly.express as px

data_dir = Path("Structured Data")
```

1. Load the events data and reshape it for analysis

Each event's `credits` field is a free-form dict mixing performer roles (`"conductor"`, `"piano soloist"`, …) with per-work composer credits (`"Composer (<work title>)"`). We split this into two long/tidy tables:

- `credits_long` — one row per (event, role, person), for performer-type roles
- `composer_credits_long` — one row per (event, work title, composer), from the `Composer (...)` keys

```
with open(data_dir / "concert_programs_events.json", encoding="utf-8") as f:
    events = json.load(f)["events"]

print(f"Loaded {len(events)} events")

events_df = pd.DataFrame([
    {
        "event_idx": i,
        "event_name": e.get("event_name"),
        "event_date": e.get("event_date"),
        "event_venue": e.get("event_venue"),
        "event_location": e.get("event_location"),
        "presenter_sponsor": e.get("presenter_sponsor"),
        "event_type": e.get("event_type"),
        "source_file": e.get("_source_file"),
    }
    for i, e in enumerate(events)
])

YEAR_RE = re.compile(r"(1[5-9]\d{2}|20\d{2})")

def extract_year(date_text):
    """Pull the first plausible 4-digit year out of a free-form date string."""
    if not date_text:
        return None
    m = YEAR_RE.search(date_text)
    return int(m.group(1)) if m else None

events_df["event_year"] = events_df["event_date"].apply(extract_year)
events_df.head()
```

```
Loaded 388 events
```

	event_idx	event_name	event_date	event_venue
0	0	Symphony Orchestra Concert	1 July 1967	Tokyo Bunka-Kaikan
1	1	Chamber Orchestra Concert	12 May 1967	Iino Hall
2	2	Wind Ensemble Chamber Music Concert	7 July 1967	Tokyo Bunka-Kaikan
3	3	Wind Ensemble Concert	20 January 1968	Beethoven Hall
4	4	Charity Concert: Haydn's 'Die Schöpfung' (The ...	4 December 1967	Tokyo Bunka Kaikan

```
HONORIFIC_RE = re.compile(
    r"^(Dr|Mr|Mrs|Ms|Miss|Herr|Frau|Fräulein|Madame|Mme|Prof|Professor)\.?\s+",
    re.IGNORECASE,
)

def strip_parenthetical(text):
    """Drop '(...)' asides -- affiliations, event notes, translations -- from a credit value
    return re.sub(r"\([^)]*\)", "", text)

def split_names(raw):
    """Split a credit value like 'Aiko Kato, Etsuko Ohya' into individual names.

    Splits on ';', ',', ' and ', ' & ' after removing parenthetical asides. Good enough for
    this corpus's "Firstname Lastname[, Firstname Lastname...]" style; doesn't handle
    'Lastname, Firstname' formatting, which doesn't appear here.
    """
    if not raw or not isinstance(raw, str):
        return []
    text = strip_parenthetical(raw)
    parts = re.split(r";|,| and | & ", text)
    return [p.strip(" .") for p in parts if p.strip(" .")]

def normalize_name(name):
    """Strip a leading honorific so 'Dr. Hans Hörner' and 'Hans Hörner' count as one person.'
    return HONORIFIC_RE.sub("", name).strip()
```

```
credit_rows = []
composer_rows = []

for i, e in enumerate(events):
    presenter = e.get("presenter_sponsor")
    for role, value in (e.get("credits") or {}).items():
        role_lower = role.lower().strip()
        if role_lower.startswith("composer"):
```

```
            work_match = re.match(r"composer\s*\((.*)\)\s*$", role, flags=re.IGNORECASE)
            work_title = work_match.group(1) if work_match else role
            if isinstance(value, str) and value.strip():
                composer_rows.append({
                    "event_idx": i,
                    "presenter_sponsor": presenter,
                    "work_title": work_title,
                    "composer_raw": value.strip(),
                    # drop trailing ", arr. X" / ", orchestrated by X" annotations for groupi
                    "composer_norm": value.split(",")[0].strip(),
                })
        else:
            for name in split_names(value):
                credit_rows.append({
                    "event_idx": i,
                    "presenter_sponsor": presenter,
                    "role_raw": role,
                    "role_lower": role_lower,
                    "person_raw": name,
                    "person_norm": normalize_name(name),
                })

credits_long = pd.DataFrame(credit_rows).merge(
    events_df[["event_idx", "event_date", "event_year", "source_file"]], on="event_idx", how=
)
composer_credits_long = pd.DataFrame(composer_rows).merge(
    events_df[["event_idx", "event_date", "event_year", "source_file"]], on="event_idx", how=
)

print(f"{len(credits_long)} performer-credit rows, {len(composer_credits_long)} composer-cred
credits_long.head()
```

```
1255 performer-credit rows, 2425 composer-credit rows
```

	event_idx	presenter_sponsor	role_raw	role_lower	person_raw	person_
0	0	Musashino Academia Musicae	conductor	conductor	Dr. Hans Hörner	Hans Hö
1	0	Musashino Academia Musicae	piano soloists	piano soloists	Masanobu Koshino	Masanob
2	0	Musashino Academia Musicae	piano soloists	piano soloists	Kunio Hirano	Kunio H
3	1	Musashino Academia Musicae	conductor	conductor	Hans Hörner	Hans Hö
4	1	Musashino Academia Musicae	violin soloists	violin soloists	Aiko Kato	Aiko Ka

2. Who were all the conductors of the Musashino Academia Musicae?

Matches any role containing "conduct" (`conductor`, `conductors`, `assistant conductor`, `conductor (Dirigent)`, `honorary life conductor`, …), so guest and assistant conductors are included alongside each concert's principal conductor. Names are grouped by `person_norm` (honorifics stripped), so "Dr. Hans Hörner" and "Hans Hörner" count as the same person; the table still shows every raw spelling encountered.

```
mus_credits = credits_long[credits_long["presenter_sponsor"] == "Musashino Academia Musicae"]

conductors = mus_credits[
    mus_credits["role_lower"].str.contains("conduct")
    & ~mus_credits["role_lower"].str.contains("being honored")
]

conductor_counts = (
    conductors.groupby("person_norm")
    .agg(
        concerts_conducted=("event_idx", "nunique"),
        raw_spellings=("person_raw", lambda s: sorted(set(s))),
        roles=("role_raw", lambda s: sorted(set(s))),
    )
    .sort_values("concerts_conducted", ascending=False)
    .reset_index()
)
print(f"{len(conductor_counts)} distinct conductors")
conductor_counts
```

```
21 distinct conductors
```

	person_norm	concerts_conducted	raw_spellings	roles
0	Antonin Kühnel	17	[Antonin Kühnel]	[conductor, conductor (
1	Asao Hasegawa	4	[Asao Hasegawa]	[conductor]
2	Masaji Kato	4	[Masaji Kato]	[conductor]
3	Sergio Sossi	4	[Sergio Sossi]	[conductor]
4	Hans Hörner	3	[Dr. Hans Hörner, Hans Hörner]	[conductor]
5	Tetsuya Sakuma	3	[Tetsuya Sakuma]	[conductor, conductors]
6	Tamotsu Maeda	3	[Tamotsu Maeda]	[conductor, conductors]
7	Tsuyoshi Sasakura	2	[Tsuyoshi Sasakura]	[conductors]
8	Ferdinand Grossmann	2	[Ferdinand Grossmann]	[conductor]
9	Hachiro Nanasawa	2	[Hachiro Nanasawa]	[conductor, conductors]

	person_norm	concerts_conducted	raw_spellings	roles
10	James Berdahl	2	[Dr. James Berdahl]	[conductor]
11	Minoru Yamada	2	[Minoru Yamada]	[conductors]
12	Robert Scholz	2	[Robert Scholz]	[conductor]
13	Tadashi Mori	1	[Tadashi Mori]	[conductor]
14	Tetsuya Oyakawa	1	[Tetsuya Oyakawa]	[assistant conductor]
15	Akeo Watanabe	1	[Akeo Watanabe]	[conductor]
16	Shigenobu Yamaoka	1	[Shigenobu Yamaoka]	[conductor]
17	Kosuke Hagiwara	1	[Kosuke Hagiwara]	[conductor]
18	Josef Zilch	1	[Josef Zilch]	[conductor]
19	Helmut Calgeer	1	[Helmut Calgeer]	[conductor]
20	Malcolm Johns	1	[Malcolm Johns]	[conductor]

```
fig = px.bar(
    conductor_counts,
    x="concerts_conducted",
    y="person_norm",
    orientation="h",
    title="Conductors of the Musashino Academia Musicae",
    labels={"concerts_conducted": "Concerts conducted", "person_norm": "Conductor"},
)
fig.update_layout(yaxis={"categoryorder": "total ascending"}, height=600)
fig.show()
```

```
Unable to display output for mime type(s): application/vnd.plotly.v1+json
```

3. Which composers did the Musashino Academia Musicae play most frequently?

Counts one credit per work performed (a concert with three pieces by three different composers contributes one count to each). `composer_norm` drops trailing annotations like ", arr. Plumpton" or ", orchestrated by …" so a work's *composer* is counted separately from whoever arranged or orchestrated it.

```
mus_composers = composer_credits_long[composer_credits_long["presenter_sponsor"] == "Musashi

composer_counts = (
    mus_composers.groupby("composer_norm")
    .size()
    .reset_index(name="times_performed")
    .sort_values("times_performed", ascending=False)
```

```
)
top_composers = composer_counts.head(20)
top_composers
```

	composer_norm	times_performed
287	W. A. Mozart	85
252	R. Schumann	83
196	L. v. Beethoven	79
173	J. S. Bach	75
87	F. Schubert	74
152	J. Brahms	74
74	F. Chopin	68
41	C. Debussy	48
134	H. Wolf	39
121	G. Verdi	34
209	M. Ravel	29
118	G. Puccini	29
81	F. Liszt	26
254	R. Strauss	26
266	S. Prokofiev	22
160	J. Haydn	22
119	G. Rossini	20
101	G. F. Handel	20
44	C. Franck	17
99	G. Donizetti	17

```
fig = px.bar(
    top_composers,
    x="times_performed",
    y="composer_norm",
    orientation="h",
    title="Most-performed composers - Musashino Academia Musicae (top 20)",
    labels={"times_performed": "Works performed", "composer_norm": "Composer"},
)
fig.update_layout(yaxis={"categoryorder": "total ascending"}, height=650)
fig.show()
```

```
Unable to display output for mime type(s): application/vnd.plotly.v1+json
```

4. Which pianists performed with which ensembles?

Matches roles containing "pian" (`piano`, `pianist`, `piano soloist(s)`, `honorary pianist`, …) but excludes `Composer (... Pianoforte ...)` work-title credits, which describe instrumentation, not a performer. "Ensemble" here is each program's `presenter_sponsor`.

```
pianists = credits_long[
    credits_long["role_lower"].str.contains("pian")
    & ~credits_long["role_lower"].str.contains("forte")
]

pianist_ensemble = (
    pianists.groupby(["presenter_sponsor", "person_norm"])
    .agg(appearances=("event_idx", "nunique"))
    .reset_index()
    .sort_values(["presenter_sponsor", "appearances"], ascending=[True, False])
)
print(f"{len(pianist_ensemble)} distinct pianist/ensemble pairs")
pianist_ensemble
```

```
75 distinct pianist/ensemble pairs
```

	presenter_sponsor	person_norm	appearances
0	American Cultural Center (Donald Albright, dir...	Roger Reynolds	1
1	Ibbs & Tillett, 124 Wigmore Street, London (ma...	Artur Schnabel	2
2	Ibbs & Tillett, 124 Wigmore Street, London (ma...	C. Bechstein Piano Co	1
3	Ibbs & Tillett, 124 Wigmore Street, London (ma...	Ltd	1
4	Lionel Powell (sole agent and manager for Mada...	Frank St. Leger	1
...	...	...	...
70	The Royal Victorian Liedertafel	Lindsay Biggins	1
71	The Royal Victorian Liedertafel	Mansley Greer	1
73	University Musical Society, University of Mich...	Mabel Ross Rhead	2
72	University Musical Society, University of Mich...	Josef Hofmann	1
74	University Musical Society, University of Mich...	Percy Grainger	1

```
fig = px.treemap(
    pianist_ensemble,
    path=[px.Constant("All ensembles"), "presenter_sponsor", "person_norm"],
    values="appearances",
    title="Pianists by ensemble (box size = concerts credited)",
```

```
)
fig.update_traces(root_color="lightgrey")
fig.update_layout(height=650)
fig.show()
```

```
Unable to display output for mime type(s): application/vnd.plotly.v1+json
```

```
# Musashino Academia Musicae dominates the dataset -- zoom in on its pianists specifically
mus_pianists = (
    pianist_ensemble[pianist_ensemble["presenter_sponsor"] == "Musashino Academia Musicae"]
    .head(20)
)

fig = px.bar(
    mus_pianists,
    x="appearances",
    y="person_norm",
    orientation="h",
    title="Pianists - Musashino Academia Musicae (top 20)",
    labels={"appearances": "Concerts", "person_norm": "Pianist"},
)
fig.update_layout(yaxis={"categoryorder": "total ascending"}, height=600)
fig.show()
```

```
Unable to display output for mime type(s): application/vnd.plotly.v1+json
```

5. Melbourne Liedertafel violinists over time

Different source file. `concert_programs_events.json` doesn't carry named violinists for the Melbourne Liedertafel — none of its 14 Liedertafel events have a performer-level `violin` credit (only work titles that happen to mention "violin", e.g. a Corelli sonata). That orchestra-roster detail exists instead in `Structured Data/concert_program_items.json`, the flat item export from `RareBooks_DataExtraction.ipynb`, which records each named performer from a printed roster page (`record_type == "performer"`). This section reads that file instead.

Current data covers two programs / two years (1893 and 1899). As more Melbourne Liedertafel programs are run through that notebook, this chart will pick them up automatically.

```
with open(data_dir / "concert_program_items.json", encoding="utf-8") as f:
    items = json.load(f)

violinists = pd.DataFrame([
    x for x in items
    if x["record_type"] == "performer"
    and x.get("manifest_organization") == "Melbourne Liedertafel"
    and "violin" in (x.get("part_or_instrument") or "").lower()
])
violinists["year"] = pd.to_numeric(violinists["manifest_date"], errors="coerce")
violinists = violinists.sort_values("year")

print(f"{len(violinists)} violinist records across {violinists['year'].nunique()} year(s)")
violinists[["year", "name", "part_or_instrument", "filename", "page_number"]]
```

```
32 violinist records across 2 year(s)
```

	year	name	part_or_instrument	filename	page_number
0	1893	MR. GEO. WESTON.	Principal Violin	UDC20260028-10.pdf	4
13	1893	2* H. Weinberg, senr.	Second Violin	UDC20260028-10.pdf	4
12	1893	E. Rawlins.	Second Violin	UDC20260028-10.pdf	4
11	1893	A. Zelman.	Second Violin	UDC20260028-10.pdf	4
9	1893	Mr, J. Wright.	Second Violin	UDC20260028-10.pdf	4
8	1893	W. Josephi.	First Violin	UDC20260028-10.pdf	4
7	1893	T. Zeplin.	First Violin	UDC20260028-10.pdf	4
10	1893	M. A. Phillips.	Second Violin	UDC20260028-10.pdf	4
5	1893	H. Schraeder.	First Violin	UDC20260028-10.pdf	4
4	1893	E. King.	First Violin	UDC20260028-10.pdf	4
3	1893	G. Sutch.	First Violin	UDC20260028-10.pdf	4
2	1893	F. Dierich.	First Violin	UDC20260028-10.pdf	4
1	1893	Mr. G. Weston.	First Violin	UDC20260028-10.pdf	4
6	1893	B. Stevens.	First Violin	UDC20260028-10.pdf	4
23	1899	Wright	violin	UDC20260028-21.pdf	4
29	1899	Stanford	violin	UDC20260028-21.pdf	4
28	1899	Nankivell	violin	UDC20260028-21.pdf	4
27	1899	Cuddon	violin	UDC20260028-21.pdf	4
26	1899	Miss Archib rid	violin	UDC20260028-21.pdf	4
25	1899	Zelman, A.	violin	UDC20260028-21.pdf	4
24	1899	Weinberg, sen.	violin	UDC20260028-21.pdf	4
22	1899	Wallenstein	violin	UDC20260028-21.pdf	4

	year	name	part_or_instrument	filename	page_number
15	1899	Brenncke	violin	UDC20260028-21.pdf	4
20	1899	Parkes	violin	UDC20260028-21.pdf	4
19	1899	Massari	violin	UDC20260028-21.pdf	4
18	1899	Hess	violin	UDC20260028-21.pdf	4
17	1899	Hunter	violin	UDC20260028-21.pdf	4
16	1899	Boy	violin	UDC20260028-21.pdf	4
30	1899	Stuckey	violin	UDC20260028-21.pdf	4
14	1899	Mr. Dierich	violin	UDC20260028-21.pdf	4
21	1899	Schieblich	violin	UDC20260028-21.pdf	4
31	1899	Whitley	violin	UDC20260028-21.pdf	4

```
fig = px.scatter(
    violinists,
    x="year",
    y="name",
    color="part_or_instrument",
    hover_data=["filename", "page_number", "source_text"],
    title="Melbourne Liedertafel violinists over time",
    labels={"year": "Year", "name": "Performer", "part_or_instrument": "Part"},
)
fig.update_layout(height=750, xaxis=dict(dtick=1))
fig.show()
```

```
Unable to display output for mime type(s): application/vnd.plotly.v1+json
```

Appendix: all credit role keys seen

Reference for extending this analysis to new roles/questions as more programs are processed. `role_lower` is the raw credit key as it appears in the source JSON (lower-cased); `sample_ensemble` is just one presenter that used it, to help spot which corpus a role tends to come from.

```
role_counts = (
    credits_long.groupby("role_lower")
    .agg(occurrences=("event_idx", "size"), sample_ensemble=("presenter_sponsor", "first"))
    .sort_values("occurrences", ascending=False)
)
role_counts
```

role_lower	occurrences	sample_ensemble
assisting artists	151	The Royal Metropolitan Liedertafel
conductor	101	Musashino Academia Musicae
president	86	The Royal Metropolitan Liedertafel
patrons	61	The Melbourne Liedertafel
piano	58	Musashino Academia Musicae
...	...	...
performer (piano)	1	American Cultural Center (Donald Albright, dir...
performer (percussion)	1	American Cultural Center (Donald Albright, dir...
performer (flute)	1	American Cultural Center (Donald Albright, dir...
performer (clarinet)	1	American Cultural Center (Donald Albright, dir...
abimelech/an old hebrew	1	University Musical Society, University of Mich...